FINAL PROJECT · DIGITAL DATABASE · 802244

MOOD WEAR

สคริปต์พูดอธิบายหน้าชั้น + ชุดคำถามจี้พร้อมบทตอบ

นางสาว ลภัสรดา หาญกุล 681310676
นายวรเศรษฐ์ บุญสวัสดิ์ 681310306
ภาคการศึกษา 2569

สคริปต์พรีเซ็น — ตามสไลด์ 10 หน้า (รวม ~5 นาที)

ผู้พูด A = นางสาวลภัสรดา · ผู้พูด B = นายวรเศรษฐ์ · พูดในเครื่องหมายคำพูดคือบทที่อ่านได้ทันที

สไลด์ 1 · ชื่อเรื่อง ผู้พูด A

"สวัสดีครับ/ค่ะ วันนี้พวกเราจะขอนำเสนอโปรเจกต์ Database Final Project รายวิชา Digital Database ระบบชื่อว่า Mood Wear ครับ/ค่ะ เราคือคนสองคน นางสาวลภัสรดา หาญกุล รหัส 681310676 และนายวรเศรษฐ์ บุญสวัสดิ์ รหัส 681310306 — Mood Wear คือระบบจัดการฐานข้อมูลร้านค้าเสื้อผ้าแฟชั่น ที่มีทั้งเว็บ Back Office ให้ทีมหลังบ้านจัดการข้อมูล และแอปหน้าร้านที่ลูกค้าใช้สั่งซื้อ ซึ่งทั้งสองฝั่งใช้ฐานข้อมูลชุดเดียวกัน"

สไลด์ 2 · ที่มาและแหล่งข้อมูล ผู้พูด A

"ทำไมถึงต้องมีสองฝั่ง? เพราะเราอยากให้เห็นภาพร้านแฟชั่นจริง — เบื้องหน้าคือร้านที่ลูกค้าเลือกซื้อ เบื้องหลังคือทีมที่ดูสต็อกและคำสั่งซื้อ"

"เรื่องแหล่งข้อมูลสำคัญมาก — ข้อมูลสินค้าทั้งหมดมาจากการ **รวบรวมเองและ web scraping** เราเก็บรายการสินค้าและรูปสินค้าสาธารณะจากเว็บแฟชั่นจริง ๆ อย่าง Samba, AMI Paris, Essentials, Human Made, Vivienne Westwood ครอบคลุม 6 หมวด จึงได้ข้อมูลสาธิตที่ใกล้เคียงความจริง"

"ตัวเลขสำคัญ: สินค้า 66 ชิ้น · ผู้ใช้ 51 คน · คำสั่งซื้อ 58 รายการ · รายการสั่งซื้อ 111 แถว · ตะกร้า 52 · ถูกใจ 52 — **ทุกตารางเกิน 50 แถวตามเกณฑ์** และข้อมูลผู้ใช้เป็นข้อมูลจำลอง ไม่มีข้อมูลจริงของบุคคลใด"

สไลด์ 3 · Conceptual Design ผู้พูด B

"ผมวิเคราะห์ระบบแล้วแยก Entity หลักได้ 5 ตัว: USER สมาชิกร้าน, PRODUCT สินค้า, ORDER คำสั่งซื้อ, CART ITEM สินค้าในตะกร้า และ FAVORITE สินค้าที่ผู้ใช้กดถูกใจ"

"ตัวเด่นที่อยากให้สังเกตคือ **ORDER ITEM** — ผมตั้งใจแยกมันออกมาเป็นตารางเชื่อม (associative entity) เพราะหนึ่งคำสั่งซื้อมีสินค้าได้หลายชิ้นหลายไซซ์ ผลลัพธ์คือจาก 5 entity กลายเป็น **6 ตาราง** ในฐานข้อมูลจริง"

สไลด์ 4 · ER Diagram ผู้พูด B

"ความสัมพันธ์มี 6 แบบ มากกว่าเกณฑ์ขั้นต่ำ 3 แบบ: users→orders และ orders→order_items, products→order_items, users/products→cart_items เป็น 1:N ทั้ง 5 แบบ และจุดสำคัญที่สุดคือ **M:N ระหว่าง USER กับ PRODUCT ผ่านการกดถูกใจ** — ผู้ใช้ถูกใจได้หลายชิ้น สินค้าชิ้นหนึ่งก็ถูกใจโดยหลายคน ซึ่งถูกแปลงเป็นตารางชื่อ favorites ที่มี composite primary key"

สไลด์ 5 · ER → Relation Schema (PK/FK) ผู้พูด B

> "แต่ละตารางแสดง PK/FK ชัดเจน — orders มี id เป็น PK และ user_id เป็น FK ไป users.id; order_items มี FK สองตัว คือ order_id ไป orders และ product_id ไป products; cart_items มี UNIQUE constraint กันใส่สินค้าเงื่อนไขเดียวกันซ้ำ"

> "จุดที่สำคัญคือ FK ของเรา**บังคับใช้จริง** — เราเปิด `PRAGMA foreign_keys = ON` ใน SQLite ทำให้ FK ถูกตรวจทั้งตอน seed และตอน Back Office ทำ Insert/Update ถ้าลบ order ที่มี order_items อยู่ ฐานข้อมูลจะ block ทันที"

สไลด์ 6 · Normalization ผู้พูด A

> "**1NF** ผ่าน — เก็บค่า atomic ไม่มีกลุ่มข้อมูลซ้ำในช่องเดียว และมี PK ทุกตาราง **2NF** ผ่าน — ตารางเดียวที่ใช้ composite key คือ favorites และไม่มี attribute ที่พึ่งพา key บางส่วน **3NF** ผ่าน — ไม่มี transitive dependency ชื่อ/ราคาสินค้าไม่ถูกเก็บซ้ำในตารางอื่น ส่วน unit_price คือ 'ราคา ณ ขณะสั่งซื้อ' เป็นเงื่อนไขธุรกิจ ไม่ใช่ dependency ต่อจากราคาปัจจุบัน"

สไลด์ 7 · Data Dictionary ผู้พูด A

> "ตัวอย่างเช่น products มี id เป็น PK, name, price, category, stock, รูป และกลุ่ม variant; order_items เชื่อม orders กับ products ด้วย FK สองตัว; users มี email เป็น UNIQUE และรหัสผ่านเก็บเป็น **hash ไม่เก็บ plain text** — ฉบับเต็มทั้ง 6 ตารางอยู่ในรายงานหัวข้อ 8"

สไลด์ 8 · System & API ผู้พูด B

> "สถาปัตยกรรมคือ แอปหน้าร้าน Flutter สื่อสารผ่าน HTTP/JSON ไปที่ Backend ชื่อ **FastAPI (Python)** และจัดการข้อมูลใน SQLite ฐานข้อมูลเดียว — Back Office ก็เป็นหน้าเว็บที่ใช้ **API ชุดเดียวกัน** ไม่แตะฐานข้อมูลข้ามทาง"

> "API หลัก: GET /products, POST /register และ /login, POST /cart และ /checkout, GET/POST/DELETE /favorites และ **GET/POST/PUT/DELETE บน /admin/t/{table}** คือ Back Office CRUD ทุกตารางครับ"

สไลด์ 9 · Back Office Demo ผู้พูด B

> "สาธิตตามปุ่ม: **แสดง/ค้นหา** — แท็บ 6 ตาราง + ช่องค้นหา + การ์ดจำนวนแถว; **เพิ่ม** — ฟอร์มแถวใหม่ บันทึกแล้วตรวจ FK; **แก้ไข** — แถวกลายเป็น input บันทึกจริงลงตาราง; **ลบ** — ยืนยันก่อนและรองรับ composite key ของ favorites คลิปแนบใช้เวลา 59 วินาที อยู่ในเกณฑ์ไม่เกิน 1 นาที"

สไลด์ 10 · สรุป + วิธีติดตั้ง ผู้พูด A

> "สรุป: ระบบมี 6 ตาราง เกินเกณฑ์ 5 entity · ความสัมพันธ์ 6 แบบ เกินเกณฑ์ 3 แบบ · ทุกตาราง ≥ 50 แถวจากข้อมูลรวบรวมเอง + web scraping · PK/FK บังคับจริง ผ่าน 3NF · Back Office CRUD ได้จริงทุกตาราง · Source Code + README อยู่บน GitHub"

> "วิธีติดตั้งบนเครื่องอื่น: `pip install -r backend/requirements.txt` แล้ว `uvicorn main:app --port 8000` ระบบจะสร้างและ seed ฐานข้อมูล fashion.db เอง เข้า Back Office ที่ `127.0.0.1:8000/admin` — ขอบคุณครับ/ค่ะ ยินดีตอบคำถาม"

ชุดคำถามจี้ของอาจารย์ + บทตอบ

เรียงตามความโหด — Q1–Q4 พื้นฐานต้องตอบทันที · Q5–Q9 จี้รายละเอียด · Q10–Q14 เหตุผลระยะยาว

Q1 ทำไมตารางเป็น 6 ทั้งที่ Entity มีแค่ 5?

เพราะ ORDER ITEM เป็นตารางเชื่อมที่แยกออกมาจากความสัมพันธ์ ORDER–PRODUCT — หนึ่งคำสั่งซื้อมีสินค้าได้ **หลายชิ้นหลายไซซ์** จึงต้องแยกเป็นตารางเอง

Q2 ผู้ใช้กับสินค้าที่ "ถูกใจ" เป็น M:N รู้ได้ยังไง?

จาก business rule — ผู้ใช้หนึ่งคนถูกใจได้หลายชิ้น และสินค้าหนึ่งชิ้นถูกใจโดยหลายคน จึงเป็น M:N และถูกแปลงเป็นตาราง **favorites**

Q3 PK ของ favorites อยู่ตรงไหน ทำไมมีสองคอลัมน์?

เป็น **composite PK (user_id, product_id)** — หนึ่งแถว = หนึ่งคนกับหนึ่งสินค้า ใช้คอลัมน์เดียวไม่ได้เพราะแต่ละฝั่งยังซ้ำได้

Q4 Normalization ผ่านถึงไหน?

ทุกตารางผ่าน **3NF** — 1NF เก็บ atomic มี PK ครบ · 2NF ไม่มี partial dependency (composite table เดียวคือ favorites) · 3NF ไม่มี transitive dependency

Q5 ⭐ unit_price เก็บราคาซ้ำ — ไม่ใช่ 3NF แล้วหรือเปล่า?

ตอบสำคัญที่สุด: unit_price คือ **ราคา ณ ขณะสั่งซื้อ (historical fact)** — ถ้า reference ไป products.price เฉย ๆ พอร้านแก้ราคา/จัดโปรฯ ใบสั่งซื้อเก่าจะบิดเบือนทันที มันไม่ใช่ transitive dependency เพราะความหมายทางธุรกิจต่างกัน

Q6 รหัสผ่านเก็บยังไง อัลกอริทึมอะไร?

เก็บเป็น **password_hash ไม่เก็บ plain text** — เรา hash ด้วย bcrypt ก่อนบันทึกตามหลักความปลอดภัยข้อมูล

Q7 บอกว่า FK บังคับจริง — จะพิสูจน์ยังไง?

เราเปิด `PRAGMA foreign_keys = ON` — ลบ order ที่มี order_items อยู่ จะได้ error **FOREIGN KEY constraint failed** ทันที ทดลองสดใน DB Browser ได้เลย และดูผลตรวจสอบรวมด้วย `PRAGMA foreign_key_check` (ผล = 0 แถว, สะอาด)

Q8 ทำไมเลือก SQLite ไม่ใช้ MySQL/PostgreSQL?

โปรเจกต์นี้เป็นเชิงสาธิต — **ไม่ต้องติดตั้งเซิร์ฟเวอร์แยก**, ยังรองรับ PK/FK/constraint ที่เราต้องใช้ครบ, และลิงก์เชื่อมระบบเดิมไป migrate ต่อได้ เพราะเรามีไฟล์ `database_export_mysql.sql` พร้อม import ใน repo — schema/API ไม่ผูกขาดกับ SQLite

Q9 ข้อมูลจาก web scraping — แค่ copy ของคนอื่นได้?

เราแยกชัด: สินค้า (ชื่อ/ราคา/หมวด/รูปสาธารณะ) มาจาก **self-collected + web scraping** ตามเกณฑ์ แล้วจัดกลุ่ม variant/หมวดเอง ส่วน users/orders/cart/favorites สร้างใหม่ทั้งหมดเพื่อจำลองระบบ — ไม่มีข้อมูลจริงของบุคคลใด

Q10 ถ้ามี 100,000 ลูกค้าสั่งซื้อพร้อมกัน SQLite ไหวไหม?

SQLite มีข้อจำกัดเรื่อง write concurrency — แต่เราเตรียม path ย้ายไป MySQL/PostgreSQL แล้ว (มี export แล้ว) โครง schema และ API รองรับการเปลี่ยน database โดยแก้แค่ connection

Q11 ผู้ใช้จะมองเห็น order ของคนอื่นได้ไหม?

Endpoint `GET /orders` กรองตาม user_id ของ session ที่ login เท่านั้น — เห็นทุกคนได้เฉพาะใน Back Office เพราะเป็นสิทธิทีมหลังบ้าน

Q12 **ลบ product ที่ถูก order อยู่ จะเกิดอะไรขึ้น?**

ฐานข้อมูล block เพราะมี FK ชี้มาจาก order_items/cart_items/favorites — จะได้ FOREIGN KEY constraint failed ใน UI เราจึงไม่ให้ลบสินค้าที่มีประวัติการขายดีกว่าปิดการขาย

Q13 **cart_items ใช้ UNIQUE ไม่ทำ composite PK เหมือน favorites — ทำไม?**

ใส่ id (auto-increment) เป็น PK เพราะแถวของ cart มี lifecycle (ต้องแก้ quantity) ไม่เหมือน favorites ที่เป็น relation แท้ ๆ — แต่กันซ้ำครบด้วย `UNIQUE(user_id, product_id, size, color)`

Q14 **ถ้าเพิ่มระบบ coupon / loyalty ต้องแก้โครงสร้างอะไร?**

เพิ่ม entity ใหม่ — COUPON (orders มี FK coupon_id, NULL ได้) และตาราง loyalty_points (FK ไป users/orders) — โครงสร้างเดิม 6 ตารางยังใช้ได้ เป็นแบบ extensible

5 จุดจี้ที่ต้องเป็นเจ้าของให้ได้

- **unit_price** ใน order_items — ตอบด้วยแนวคิด "ราคาบันทึกขณะสั่งซื้อ" (Q5)
- **Composite PK ของ favorites** — ตอบว่า relation จริงไม่มี id ยิงเดี่ยว (Q3, Q13)
- **FK บังคับจริง** — เตรียมโชว์ `PRAGMA foreign_keys` + ลบเออร์เรอร์จริง ใน DB Browser (Q7, Q12)
- **เหตุผลเลือก SQLite หรือหลักฐาน migrate ต่อ** (Q8, Q10)
- **เดโม่จริง** — ต้องเปิด Back Office + DB Browser โชว์ได้หน้าชั้น

ประโยคครับมือถ้าไม่รู้คำตอบ

สถานการณ์	ประโยคที่พูดได้
ไม่แน่ใจข้อมูลเฉพาะ	"ข้อมูลเชิงรายละเอียดอยู่ในรายงานหัวข้อ 8 ครับ — ขออนุญาตเปิดแล้วตอบตามจริง"
ไม่รู้รายละเอียดจริง	"ยังไม่ได้ implement ครับ แต่ตามหลักที่เราเรียน X คือ ... ถ้าให้ออกแบบผมจะทำด้วยวิธี ..."
รู้ว่าตัวเองผิด	"ตรงนี้เป็นจุดอ่อนจริงครับ ถ้าพัฒนาต่อผมจะปรับเป็น ..."

เทคนิคในการพรีเซ็น

- ถ้าเวลาน้อยตัดสไลด์ 5, 6, 7 ให้ย่อเหลือ 1 ประโยค: "PK/FK บังคับจริงทุกตาราง และผ่าน 3NF ครับ"
- ไฟล์คิวรีสาธิต `backend/demo_queries.sql` พร้อมจะเปิดใน DB Browser ถ้าอาจารย์ขอดู query จริง
- มีหน้าโชว์เลข: สินค้า 66 · ผู้ใช้ 51 · คำสั่งซื้อ 58 · รายการสั่งซื้อ 111 · ตะกร้า 52 · ถูกใจ 52 (สไลด์ 2 + Back Office หน้าแรก)

ดาวน์โหลด: Source Code จาก GitHub (mood-wear) · Back Office ที่ http://127.0.0.1:8000/admin · ภาคการศึกษา 2569